

CHAPTER ONE

INTRODUCTION

1.1 Background of the Study

Legacy University, Okija, sits on the Onitsha-Owerri Expressway in Anambra State and has been running since 2016. Like a good number of relatively young Nigerian private universities, most of its day-to-day administrative work still happens the way it would have happened twenty years ago: registration forms filled out by hand or emailed as spreadsheets, results collated in departmental offices before they ever reach a student, and departmental records kept wherever the person in charge of them happens to keep them. The university's public website gives prospective students information about the institution, but it stops there. There is no student portal, no staff portal, nothing that lets a registered student log in and see their own course list or a lecturer submit a result without physically handing over a sheet of paper.

This is not unusual for a young institution still building out its administrative infrastructure, but it creates a specific kind of friction that shows up most sharply during registration periods and whenever results are due. A registrar's office working from spreadsheets has no easy way to guarantee that only the right person edited a given student's record, or to prove after the fact who changed what. A student who wants to check whether their registration went through has to ask someone, in person or by phone, rather than simply looking. None of this is catastrophic on its own, but it adds up to a system that is slower than it needs to be and harder to audit than it should be.

Role-Based Access Control (RBAC) is a well-established answer to the access-control half of that problem. Rather than everyone with a login being able to see or touch everything, RBAC ties what a user can do to the role they hold, so a student's account is mechanically incapable of editing another student's results no matter what URL they type into the browser. Ferraiolo and Kuhn's early formalization of the model, and the NIST RBAC standard that grew out of Sandhu et al.'s later work, are still the reference point most practical systems build from, and Django, the framework this project is built on, already ships with a group-based permission system that maps onto RBAC reasonably well.

This project takes that model and applies it to Legacy University's specific situation: a working web portal, built with Python and Django, that gives each class of user (students, lecturers, heads of department, the registrar, and so on) a dashboard scoped to what they actually need to do, backed by a database that enforces those boundaries on the server rather than trusting the browser to hide the parts a user shouldn't see.

1.2 Problem Statement of the Study

Legacy University currently has no functional system for managing academic records, course registration, or results processing. Everything runs on paper, spreadsheets, or informal digital tools with no consistent rule governing who is allowed to view or change a given piece of institutional data, and a few distinct problems fall out of that.

There is no reliable access boundary to start with. Anyone with access to a shared spreadsheet can, in principle, see or edit records that have nothing to do with them, and nothing forces a lecturer's changes to stay within their own courses or a student's view to stay within their own record.

The processes themselves are also slow. Course registration and result checking both depend on someone being available to process a form or answer a query, which means students wait, and staff spend time on repetitive administrative work that a system could handle on its own.

Worse, because of the first two problems, there is effectively no record of who did what. If a result is entered incorrectly or a registration goes missing, there is rarely a clean way to trace how it happened, since the underlying tools were never built with that kind of accountability in mind.

1.3 Aim and Objective of the Study

Aim: to design and implement a secure, web-based University Information Management Portal for Legacy University, built around role-based access control, that replaces the manual processes described above for the modules within its scope.

Objectives:

1. To design a role-based access control model covering the university's key user roles (Student, Lecturer, Head of Department, Registrar, Bursar, Dean, and IT Administrator) and enforce it at the application layer.
2. To implement a secure authentication system, including a passwordless first-login flow for students (verified by an emailed PIN) and an emailed setup link for staff, so no account ever starts life with a shared default password.
3. To build a student information module covering profile management, self-service account changes, and department/faculty administration.
4. To build a course registration module that lets students register for courses scoped to their department and level, with validation against the department's programme structure.
5. To build a results and grading module that lets lecturers enter scores against their assigned courses, with automatic grading and GPA/CGPA computation on the NUC's five-point scale.
6. To evaluate the resulting system against the requirements set out in the accompanying Software Requirements Specification, and to document where the final build differs from the original design.

1.4 Significance of the Study

For students, the portal removes the need to physically track down staff to register for a course or find out whether a result has been published; both become something they can check for themselves, at any time, from their own account.

For the registrar's office and departmental staff, the system centralizes records that currently live across spreadsheets and paper files, and it removes a category of error that comes from copying the same information between different manual records.

For the Department of Computer Science, the project is a working demonstration that a role-based access control system, correctly scoped, is something a single student developer can design and build within a final year project's timeline. That has some bearing on whether a wider, institution-backed version of the idea is worth pursuing later, which is the question the accompanying pilot proposal puts to the department directly.

For the researcher, the project is a chance to work through the practical gap between how RBAC is described in the literature and what it actually takes to enforce it in a real, running application. Some of that gap turned out to be smaller than expected, and some of it, documented honestly in Chapter Three and Chapter Four, turned out to be larger.

1.5 Scope of the Study

The system covers four modules, in line with the scope agreed with the department before implementation began. Authentication and role-based access control handle login, session management, account lockout, and the per-role permission checks that govern every other module. Student information brings together student profiles, department and faculty records, and self-service account management, including preferred usernames, password recovery, and email changes. Course registration lets students see course listings scoped to their department and level and register for the current semester, and results and grading covers score entry by lecturers, automatic grading on the NUC five-point scale, and GPA/CGPA computation for students.

Seven roles are modelled in the RBAC layer: Student, Lecturer, Head of Department, Registrar, Bursar, Dean, and IT Administrator. Not every role has a fully built-out feature set behind it; the Bursar role, for instance, exists as a real, working login role with its own dashboard, but no fee-management functionality sits behind it in the current build. Fee management, transcript processing, attendance tracking, and system-wide audit logging were all part of the original seven-role, nine-module design discussed in Chapter Three, but none of them made it into the final implementation. Where that matters for how a chapter should be read, it is flagged explicitly rather than left for the reader to discover on their own.

1.6 Limitations of the Study

The system was built and tested by a single developer within the time available for a final year project, which set a hard ceiling on how much of the originally-planned scope could realistically be delivered; Section 1.5 above and Chapter Three's discussion of scope evolution both go into what was cut and why.

The database in use is SQLite, chosen for its zero-configuration convenience during development. No PostgreSQL configuration was ever added, so the "production-ready" database swap discussed in some of the project's earlier planning documents did not happen in practice.

The system has not been deployed to a live production environment or trialled with real institutional data; it was built and tested against realistic but synthetic student, course, and result records. The accompanying pilot proposal to the department outlines what a real, limited-scope trial would look like, but that trial had not started as of this report.

No payment gateway is integrated, since fee management was outside the final scope. The system is browser-based only, with no native mobile application.

1.7 Definition of Terms

RBAC (Role-Based Access Control): a security model in which a user's ability to view or act on data is determined by the role or roles assigned to their account, rather than by permissions attached to the individual user.

MVT (Model-View-Template): the architectural pattern Django applications follow, with a Model layer for data, a View layer for request handling and business logic, and a Template layer for rendering HTML.

CRUD: shorthand for the four basic data operations a system supports: Create, Read, Update, Delete.

HOD: Head of Department, one of the seven roles modelled in the system.

CSRF (Cross-Site Request Forgery): an attack that tricks a logged-in user's browser into submitting a request they didn't intend to make; Django's CSRF middleware protects every form in the portal against it.

FR: Functional Requirement, as used throughout the Software Requirements Specification (Appendix, or see docs/SRS_Legacy_University_Portal.md) and referenced by ID (e.g. FR-AUTH-05) where a design decision ties back to a specific requirement.

NUC: the National Universities Commission, the Nigerian regulatory body whose five-point grading scale (A through F, with grade points 5 down to 0) the results module implements.

GPA / CGPA: Grade Point Average and Cumulative Grade Point Average, the semester and running-total measures of a student's academic performance computed from their graded results.